
Visión General del Ciclo de Vida del Software - Parte 1

INFO 248

Dra. Valeria Henríquez

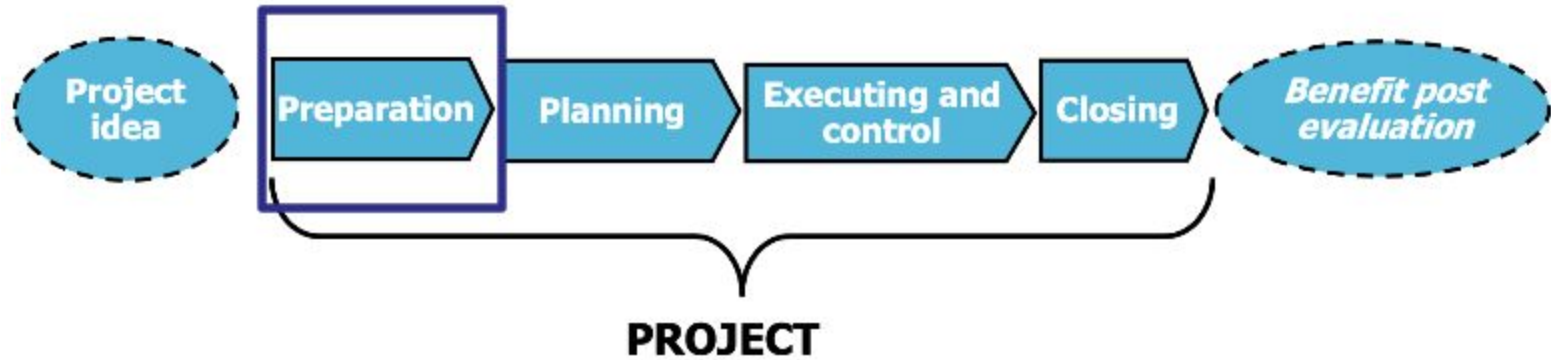
Quiz

<https://forms.gle/ze3DcwVRbFMTg5SC9>



Visión General del Ciclo de Vida

Fases de la Gestión de Proyectos



Fase de Preparación

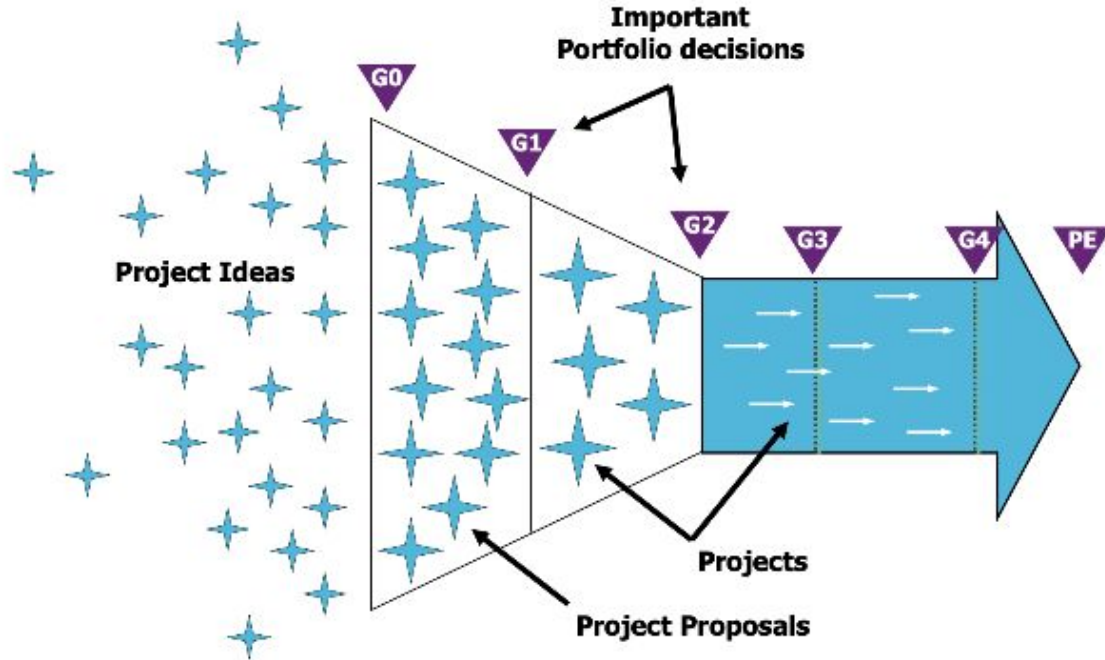


Objetivo principal: decidir si el proyecto se lleva a cabo o no. Se analiza:

- Necesidad del proyecto
- Valoración económica (rentabilidad)
- Viabilidad de la idea

No es tarea del jefe del proyecto (de hecho, probablemente aún no esté asignado) si no de la Dirección de la compañía como parte de su gestión estratégica

Fase de Preparación



Fase de Preparación

Dos “productos” principales:

Project charter (cartera del proyecto): documento que autoriza formalmente la existencia de un proyecto y confiere al director del proyecto la autoridad para asignar los recursos de la organización a las actividades del proyecto.

- Convierte una idea abstracta en un objetivo concreto
- Justifica la necesidad del proyecto
- Incluye una estimación de beneficios financieros
- Incluye una estimación de requisitos preliminares

Estudio de viabilidad: permite a las organizaciones determinar si un proyecto tiene sentido desde el punto de vista **económico** y **operativo** para determinar si el proyecto puede ser ejecutado.

Project Charter Example			
Project Name	IVR Project		
Project Sponsor	Dave Sponsor	Project Manager	Alice Michaels
Date of Project Approval	8th Mar 2015	Last Revision Date	17th Apr 2015
Project Description	To introduce a new automated telephone system to ensure all calls get answered.		
Scope	A IVR system will be introduced to assist the sales team in taking orders, and also to ensure no orders are missed. The system is only to help the sales team at this stage, other teams such as support are out of scope.		
Business Case	To increase orders per sales team member by 20% from current levels. To reduced unhandled calls to 0%. To increase customer satisfaction by 10 points.		
Constraints (in priority order)	Time	4 months	
	Budget	4 developers + 1 sales team rep	
	Scope	TBD	
	Quality	Prioritize time & budget over quality	
Project Deliverables	An IVS system to assist the sales team + training for the sales team + support during the first operational month of the system.		
Benefits (measurable results)	See KPIs below + business case above		
	KPI	Baseline	Goal
	Orders per sales person pd	20	24
	Unhandled calls pd	11	0
	Customer satisfaction	17	27
Steering Committee	CEO	Project Team	Sales Rep
	Finance Director		4x developers TBD
	Sales Director		
Key Stateholders	Name	Success Criteria	
Risks	No team members have any previous experience of IVR setup, so there is a chance we've hugely underestimated the work involved.		

Ejemplo Project Charter

1. ¿Qué se desarrollará?
2. ¿Por qué?
3. ¿Con qué presupuesto?
4. ¿En cuánto tiempo?

Fuente: The Digital Project Manager
<https://thedigitalprojectmanager.com/project-charter/>

Estudio de Viabilidad



Viabilidad económica



Viabilidad técnica



Viabilidad temporal

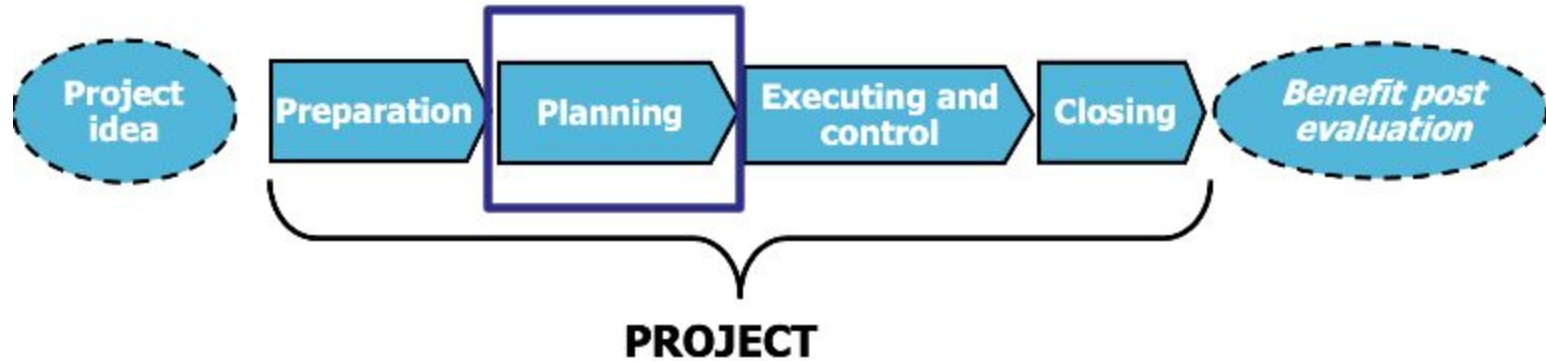


Viabilidad operativa

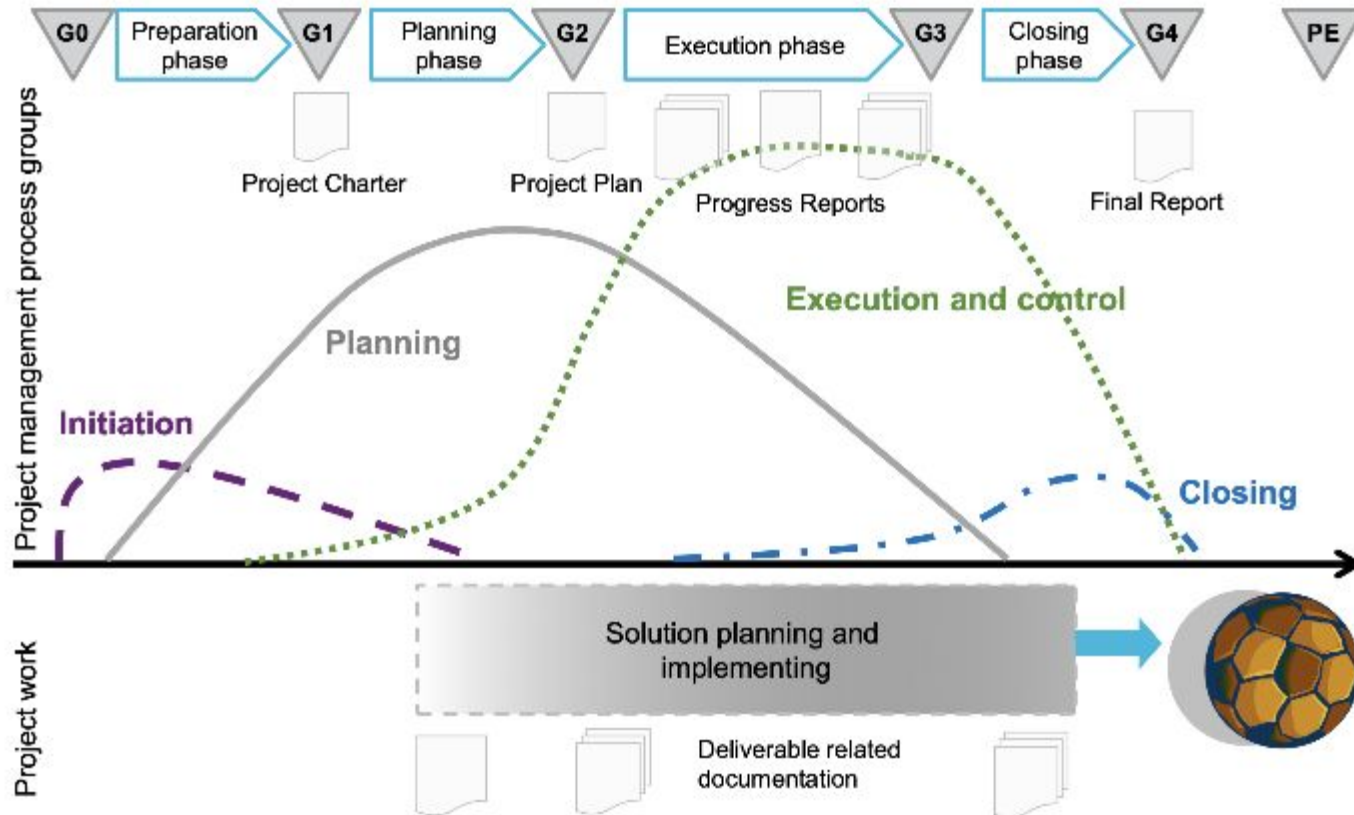


Viabilidad legal

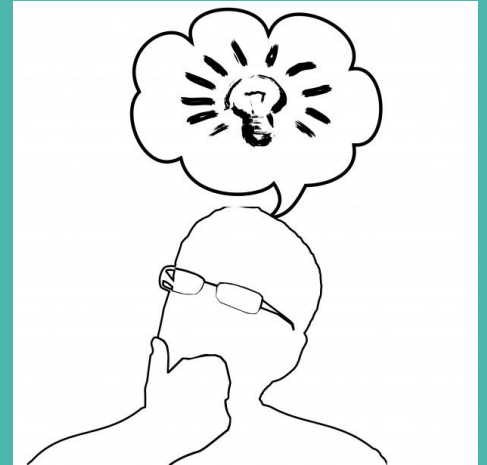
Fases de la Gestión de Proyectos



Planificación de Proyectos



¿Qué hacemos cuando
planificamos algo?



Planificación de Proyectos

Selección de una estrategia para producir un producto, así como la definición de las actividades a realizar para conseguir ese objetivo, la concurrencia y solapamiento de dichas actividades y los recursos asignados.

- ¿Qué actividades tengo que hacer?
- ¿Cuándo las voy a hacer?
- ¿Cuánto me va a costar hacerlas? ¿Qué recursos necesito? En términos de personal, tiempo, recursos económicos, bloqueo de
- recursos, etc.
- ¿Quién es el responsable de cada tarea?

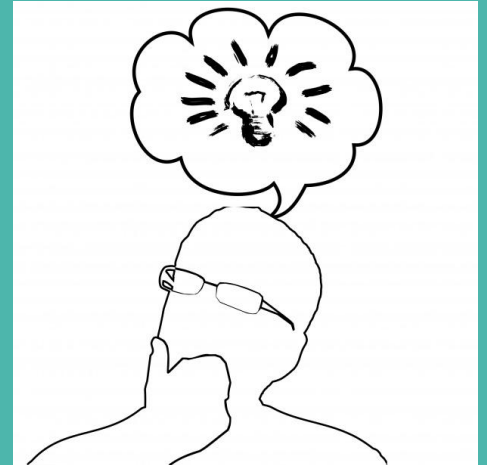
Planificación de Proyectos

Selección de una estrategia para producir un producto, así como la definición de las actividades a realizar para conseguir ese objetivo, la concurrencia y solapamiento de dichas actividades y los recursos asignados.

- 
- ¿Qué actividades tengo que hacer?
 - ¿Cuándo las voy a hacer?
 - ¿Cuánto me va a costar hacerlas? ¿Qué recursos necesito? En términos de personal, tiempo, recursos económicos, bloqueo de recursos, etc.
 - ¿Quién es el responsable de cada tarea?

**Modelos de Ciclo
de Vida y Procesos**

**¿Qué actividades/tareas se
llevan a cabo en la ejecución
de un proyecto software?**





Identificación y
análisis de requisitos



Diseño y arquitectura



Programación



Pruebas



Integración

Mantenimiento
Documentación
Soporte al cliente
Gestión del proyecto

...

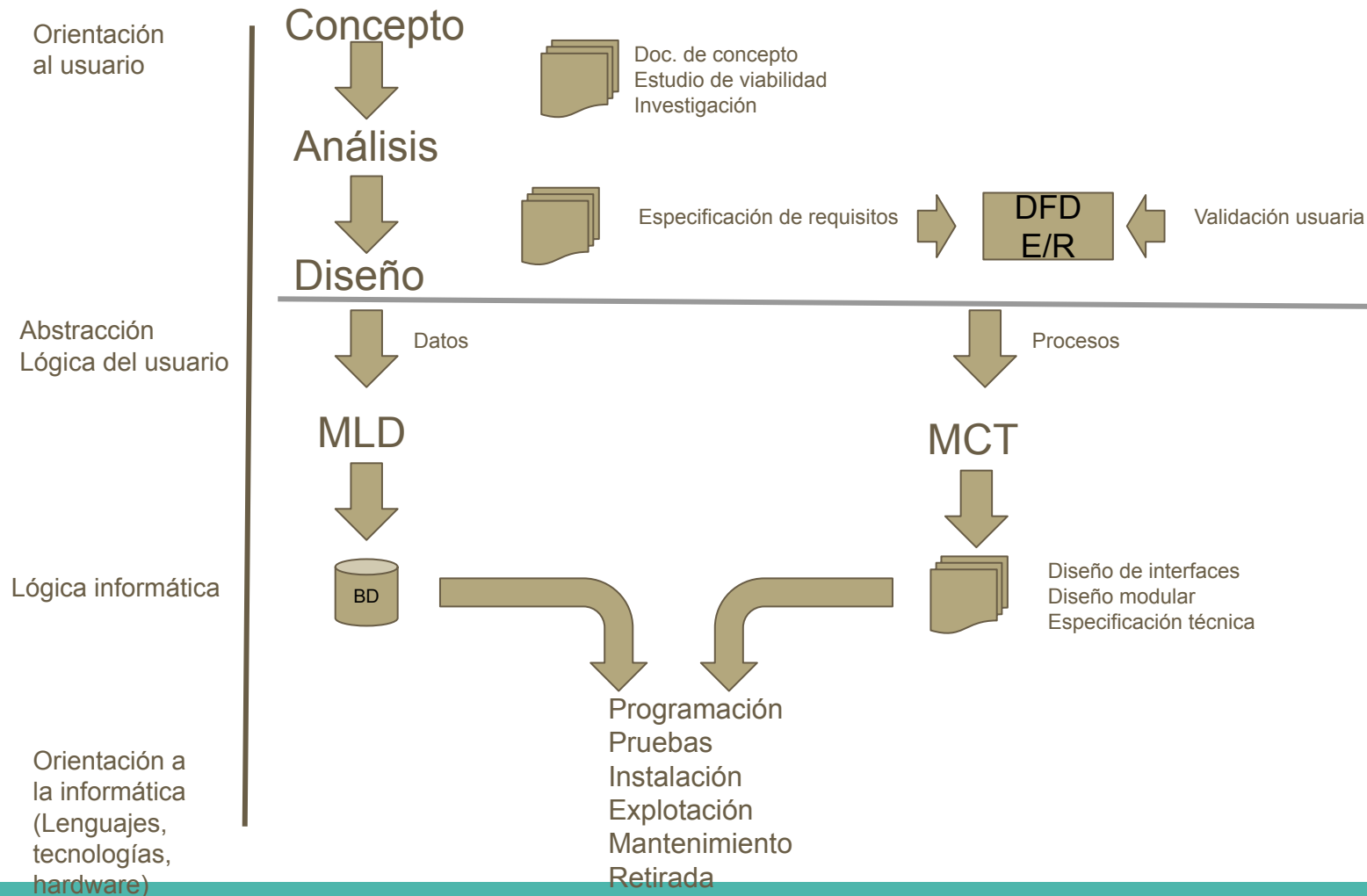
¿Qué es un Ciclo de Vida? ¿Y un Modelo de Procesos?

Ciclo de Vida

Evolución de un sistema, producto, servicio, proyecto u otra entidad, desde su concepción hasta el retiro.



Ejemplo ciclo de vida de una casa

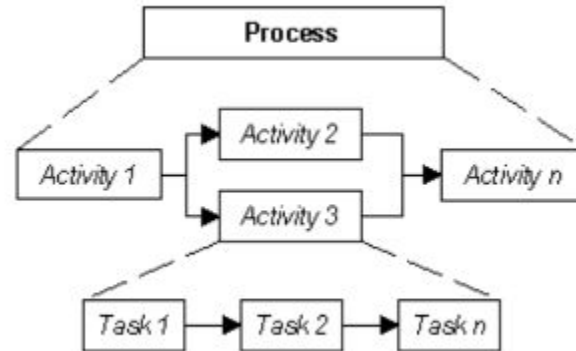


Ciclo de Vida

La función principal de un ciclo de vida software es determinar el orden de las fases así como de los procesos involucrados en el desarrollo de software, teniendo en cuenta el modelo que se haya utilizado como referencia.

Marco de referencia que contiene:

- Fases/Etapas
 - **Procesos**
 - Actividades
 - Tareas



Modelo de Procesos (de Referencia)

Describen los procesos que podrían utilizarse para adquirir, suministrar, desarrollar, explotar, soportar y mantener el software, es decir desde que surge la idea hasta que se retira.

- ISO/IEC 12207-1 Proceso del Ciclo de Vida Software
- CMMI ® Model (Capability Maturity Model ® Integration)

ISO 12207

Procesos principales

CUS.1 Adquisición

Preparación de adquisición
Selección del suministrador
Supervisión del suministrador
Aceptación del cliente

CUS.2 Suministro

CUS.4 Explotación

Uso operativo
Soporte del cliente

CUS.3

Educción de requisitos

ENG.1 Desarrollo

Análisis y diseño de los requisitos del sistema	Construcción software
Análisis de los requisitos software	Integración software
Diseño software	Pruebas del software
	Integración y pruebas del sistema

ENG.2 Mantenimiento del sistema y software

Procesos de soporte

SUP. 1 Documentación

SUP.2 Gestión de configuración

SUP.3 Aseguramiento de calidad

SUP.4 Verificación

SUP.5 Validación

SUP.6 Revisión conjunta

SUP.7 Auditoría

SUP.8 Resolución de problemas

Procesos de la organización

MAN. 1 Gestión

MAN.2 Gestión del proyecto

MAN.3 Gestión de la calidad

MAN.4 Gestión del riesgo

ORG. 1 Alineamiento con la organización

ORG.2 Mejora

Establecimiento, Evaluación y Mejora del proceso

ORG.3 Gestión de recursos humanos

ORG.4 Infraestructura

ORG.5 Medida

ORG. 6 Reutilización

Modelos de Proceso

□ No imponen:

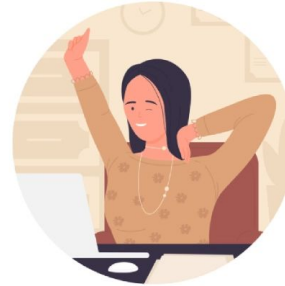
- La utilización de un Modelo de Ciclo de Vida específico
 - El uso de una Técnicas / Herramientas software específicas
 - Una estructura de organización para un proyecto de desarrollo software
- Cada empresa debería seleccionar, para cada proyecto, los procesos que considere necesarios realizar (con las actividades que crea convenientes) y establecer sus propios CV.

Diferencia entre Modelo de Procesos y Ciclo de Vida

Un Modelo de Procesos determina qué **procesos pueden utilizarse para desarrollar software**.

El Ciclo de Vida determina **cuáles son los procesos y en qué orden se realizan**, y cuáles son las actividades y/o tareas implicadas que voy a utilizar en mi desarrollo concreto.

5 Min. de Pausa

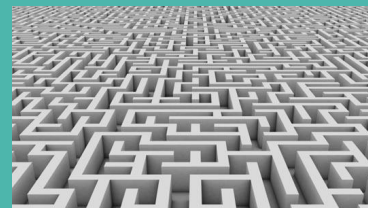


1960s Desarrollo Software ≠ Desarrollo Hardware

- El software es más fácil de modificar/reproducir que un producto hardware (y menos costoso) □ codificar y corregir.

- **Sin embargo, el software es invisible**

- No se pueden tomar medidas físicas (por ejemplo, peso)
- La fiabilidad del software no se puede medir de forma exacta como en el caso de productos hardware.
- Muchos más estados, modos y rutas para probar, lo que hace que sus especificaciones sean mucho más difíciles de probar.



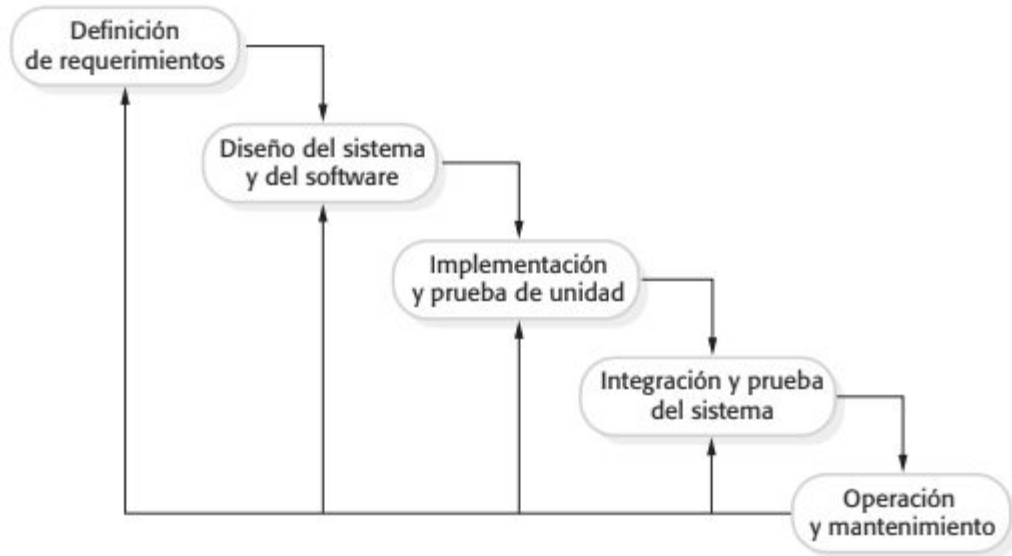
- **Codificar y corregir** □ problemas de mantenimiento, problemas de calidad, etc. □ **usuarios descontentos.**

Dijkstra: “Las pruebas sólo pueden demostrar la presencia de errores, no su ausencia”.

1970-1990: Formalidad

- **Reacción al enfoque codificar y corregir.**
 - Se necesitaban métodos mejor organizados y prácticas más disciplinadas para escalar los productos y proyectos software que cada vez eran más grandes.
- **Comienzo del estudio de la Ingeniería del Software tal y como la conocemos en la actualidad.**
- **Ciclo de Vida dividido en fases:** la fase de codificación está mucho más organizada y precedida por una fase de diseño y otra de requisitos, y seguida por una fase de pruebas.
- Modelos de ciclo de vida **“pesados” (tradicionales)**, muy estructurados y estrictos (plan-driven).

Ciclos de vida predictivos: Cascada



- Secuencia uniforme y ordenada de los pasos (fases) de desarrollo.
- Cada fase empieza cuando ha terminado la anterior.
- Para pasar de una fase a otra es necesario conseguir todos los objetivos de la etapa previa.

Modelo de Ciclo de Vida en Cascada: 1970 - Formalidad

Ventajas:

- Fácil de planificar, comprender y seguir.

Desventajas:

- Inflexible.
- Exige definición completa de requisitos al comienzo.
- No se adapta a cambios en funcionalidad y tecnología (dificultad para el usuario para establecer todos los requisitos al principio).
- Lento y costoso – requiere mucha documentación y un proceso muy estricto de desarrollo (hasta que no se finalice una fase no se pasa a la siguiente).
- Producto disponible al final del desarrollo del proyecto.

Se recomienda este ciclo de vida cuando:

Los requisitos son estables y están bien comprendidos al comienzo del desarrollo. El usuario prefiere todas las funcionalidades en la primera entrega (o las entregas parciales no reportan beneficio).

1980: Productividad y Escalabilidad

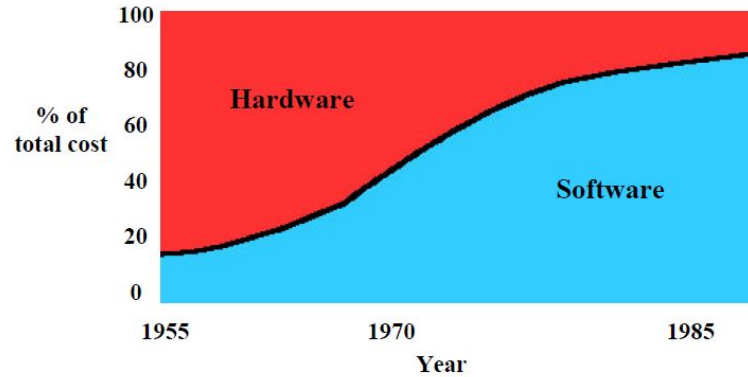
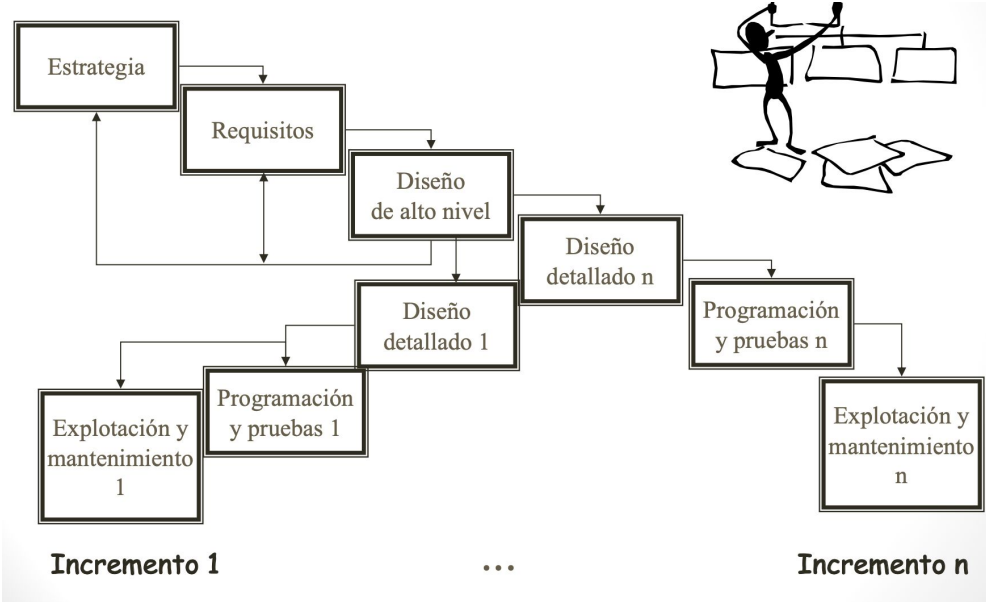


Figure 5. Large-Organization Hardware-Software Cost Trends (1973)

- Problemas para escalar los ciclos de vida en cascada.
- Modelo de ciclo de vida muy costoso y lento (“pesado”).

Modelo de Ciclo de Vida Incremental



- Permite una secuencia no lineal en el desarrollo.
- En la planificación inicial se determinan qué funcionalidades se llevarán a cabo en cada incremento y cuándo.
- Se va creando el sistema software añadiendo componentes funcionales al diseño llamados incrementos.
- En cada incremento se actualiza el sistema con implementación de nuevos requisitos.

Modelo de Ciclo de Vida Incremental

- + Integración de resultados sucesivos obtenidos después de cada iteración.
- + Más flexible, no se necesita un conjunto exhaustivo de requisitos al comienzo.
 - Aún existe el problema de si los requisitos son válidos. Los errores en los requisitos se detectan tarde y, por tanto, corregirlos sigue siendo costoso.
 - No tiene éxito si no hay una implicación del usuario.
 - Requiere una gestión de la configuración mucho más avanzada .

Se recomienda este ciclo de vida cuando:

- Los requisitos están bien comprendidos al comienzo del desarrollo.
- El usuario se beneficia de obtener versiones intermedias.

Modelo de Ciclo de Vida Iterativo/Evolutivo



- Las necesidades no están bien definidas.
- Decisión consciente de retrasar una porción del producto software final.
- Las evoluciones del sistema no están previstas de manera incremental por lo que hay una vuelta a planificar y diseñar después de cada evolución.

Incremental vs Iterativo/Evolutivo



Modelo de Ciclo de Vida Iterativo/Evolutivo

- + No se necesitan conocer todos los requisitos al comienzo .
- + Al igual que el incremental los entregables intermedios facilitan la retroalimentación por parte del cliente.
- Es difícil estimar el esfuerzo inicial necesario.
- Es difícil también medir el progreso.
- No tiene éxito si no hay una implicación del usuario.
- Riesgo de no acabar nunca si se están continuamente incluyendo nuevas versiones del producto y mejoras.

Se recomienda cuando los requisitos o el diseño no están completamente definidos al comienzo del desarrollo y es posible que haya grandes cambios. También cuando se están probando nuevas tecnologías.

¿Cómo complementar lo visto en clases?

Capítulo 2 del libro Sommerville

Nos vemos la próxima clase